Universidad Tecnológica de Honduras

ASIGNATURA:
PROGRAMACION MOVIL I

TRABAJO:
Tarea 2.4

CATEDRÁTICO:
Ricardo Lagos

REALIZADO POR:

José Fernando Orellana Reyes (202210010352)
Carlos Roberto Cardenas Martínez (202430010599)
Jennifer Mariela Marquez Chacon (202330010411)
Michael Abad Diaz Castro (202310040165)
Fernando Abad Diaz Castro (202410040257)
Romel Donaldo Barahona Márquez (202410010359)

REALIZADO POR:
Repositorio: github.com/Kirklen0910/Exa1Pa1199

FECHA:
13 noviembre de 2026

CONTENIDO

1. Introducción
2. Arquitectura de seguridad
3. Usuarios y contraseñas
4. Endpoints de autenticación
5. Configuración del token JWT
6. Protección de la clave de firma
7. Autorización por roles y propietario
8. Respuestas 401 y 403
9. HTTPS y documentación en Swagger
10. Sesión en la aplicación Android
11. Análisis del token decodificado
12. Pruebas de seguridad
13. Evidencias de la aplicación
14. OAuth 2.0 con código de autorización y PKCE
15. Distribución del trabajo
16. Conclusiones

1. INTRODUCCIÓN

En las tareas 2.2 y 2.3 se construyó la API REST de la Bitácora de campo y su cliente Android. Hasta ese momento cualquier persona que conociera la dirección del servidor podía consultar, crear, modificar o eliminar registros. En esta tarea se protegió la solución completa.

La API ahora administra usuarios con contraseñas protegidas mediante hash, emite tokens JWT firmados con vencimiento de diez minutos y aplica autorización por rol y por propietario del registro. La aplicación Android incorpora inicio de sesión, creación de cuenta, almacenamiento cifrado de la sesión, envío automático del token y manejo diferenciado de las respuestas 401 y 403. Finalmente, se explica cómo funcionaría el flujo OAuth 2.0 con código de autorización y PKCE si la autenticación se delegara a un proveedor externo.

2. ARQUITECTURA DE SEGURIDAD

La seguridad se agregó respetando la arquitectura limpia de la API. El dominio define las entidades Usuario y Roles; la capa de aplicación define los contratos IHashContrasena, IGeneradorToken e IUsuarioRepository y los servicios AuthService y RegistroService; la infraestructura implementa el hash, la generación del token y la persistencia; y la capa Api configura la autenticación JWT Bearer, la autorización y las respuestas de error. La Figura 1 muestra el recorrido de una sesión completa.

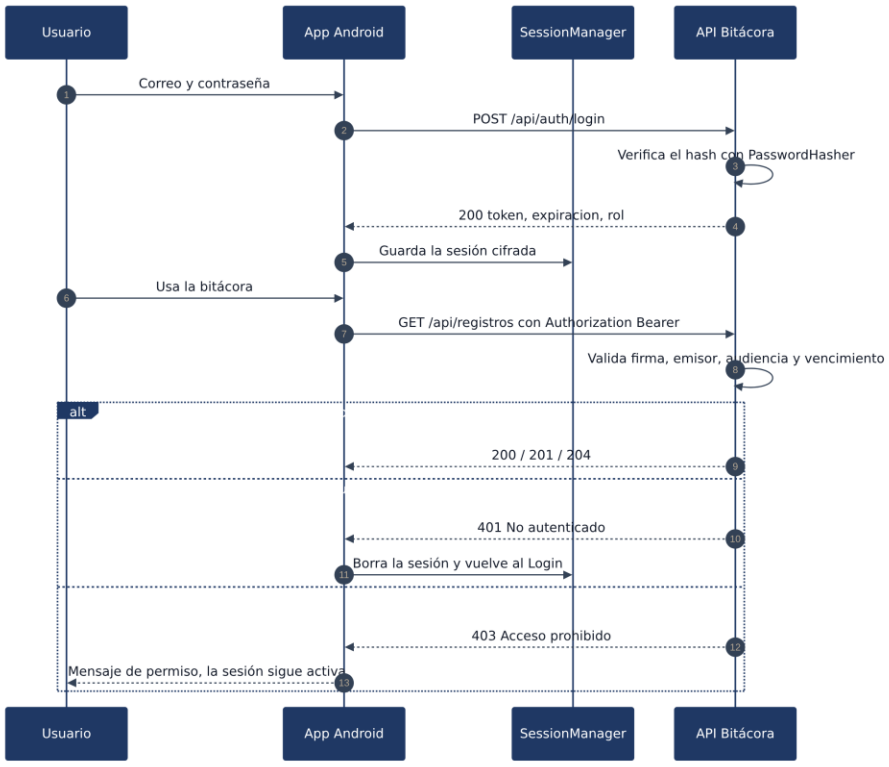


Figura 1. Flujo de inicio de sesión, envío del token y respuestas 401 y 403

Tabla 1. Componentes de seguridad por capa

Capa	Componentes	Responsabilidad
Domain	Usuario, Roles, Registro (propietario)	Modelo del usuario, roles Admin y Usuario y dueño de cada registro.
Application	AuthService, RegistroService, UsuarioActual, ResultadoOperacion	Reglas de registro, inicio de sesión y verificación del propietario.
Infrastructure	HashContrasena, GeneradorTokenJwt, JwtOpciones, UsuarioRepository, migración UsuariosYPropietario	Hash de contraseñas, firma del token y tablas de usuarios.
Api	AuthController, RegistrosController, RespuestasSeguridad, SeguridadSwaggerFilter, Program.cs	Endpoints, atributos de autorización, 401/403 y Swagger.
Android	SessionManager, AuthInterceptor, AuthRepository, LoginActivity, SesionActivity	Sesión cifrada, envío del token y control de la sesión.

3. USUARIOS Y CONTRASEÑAS

Se creó la entidad Usuario con correo único, hash de la contraseña, rol y fecha de creación, y la migración UsuariosYPropietario agregó la tabla de usuarios y la columna del propietario en cada registro.

- **Hash seguro.** Las contraseñas nunca se guardan en texto plano. HashContrasena utiliza PasswordHasher de ASP.NET Core Identity, que aplica PBKDF2 con sal aleatoria por usuario, por lo que dos contraseñas iguales producen hashes distintos.
- **Roles.** Toda cuenta nueva recibe el rol Usuario. El rol Admin no se puede solicitar desde la API; la cuenta administradora se crea al iniciar la API con el correo y la contraseña guardados en los secretos de usuario.
- **Mensaje genérico.** Si el correo no existe o la contraseña es incorrecta, la respuesta es la misma: 401 con «El correo o la contraseña no son correctos.». Además, cuando el correo no existe se verifica un hash ficticio para que el tiempo de respuesta no revele qué correos están registrados.

4. ENDPOINTS DE AUTENTICACIÓN

Los endpoints de autenticación son públicos ([AllowAnonymous]); los de registros exigen un token válido ([Authorize]).

Tabla 2. Endpoints y respuestas

Endpoint	Acceso	Respuestas
POST /api/auth/registro	Público	201 cuenta creada · 400 datos inválidos · 409 correo ya registrado
POST /api/auth/login	Público	200 token, expiración, rol y correo · 401 credenciales inválidas

Endpoint	Acceso	Respuestas
GET /api/registros y GET /api/registros/{id}	Autenticado	200 · 401 sin token o token inválido · 404 no existe
POST /api/registros	Autenticado	201 y el registro queda a nombre del usuario del token
PUT /api/registros/{id}	Propietario	204 · 403 si el registro pertenece a otro usuario
DELETE /api/registros/{id}	Solo Admin	204 · 403 si el usuario tiene rol Usuario

5. CONFIGURACIÓN DEL TOKEN JWT

GeneradorTokenJwt crea el token con los datos de JwtOpciones y Program.cs configura su validación en cada petición. La Tabla 3 resume los parámetros.

Tabla 3. Parámetros del token

Parámetro	Valor	Propósito
Emisor (iss)	Bitacora.Api	Solo se aceptan tokens emitidos por esta API.
Audiencia (aud)	Bitacora.App	El token está destinado a la aplicación de la bitácora.
Algoritmo	HMAC SHA-256 (HS256)	Firma simétrica; se rechaza cualquier otro algoritmo.
Clave de firma	Mínimo 32 bytes, en user-secrets	Garantiza que el token no fue alterado.
Vencimiento	10 minutos	Limita el tiempo de uso de un token robado.
ClockSkew	Cero	El token vence exactamente a los diez minutos, sin tolerancia.
Claims	sub, email, role, jti	Identifican al usuario, su rol y el token.

La validación revisa emisor, audiencia, firma, presencia de la fecha de expiración y vigencia. NameClaimType se asignó a email y RoleClaimType a role, para que [Authorize(Roles = "Admin")] lea el rol directamente del token.

6. PROTECCIÓN DE LA CLAVE DE FIRMA

La clave Jwt:Key no aparece en el código, en appsettings.json ni en el repositorio. Se guarda con los secretos de usuario de .NET, que se almacenan fuera de la carpeta del proyecto, en el perfil del equipo de cada desarrollador.

- **Desarrollo.** Cada integrante configura su propia clave con dotnet user-secrets set "Jwt:Key" y, de la misma forma, Admin:Correo y Admin:Password.
- **Validación al iniciar.** Si la clave falta o tiene menos de 32 bytes, la API se detiene con un mensaje que indica cómo configurarla, en lugar de arrancar con una clave débil.

- **Producción.** La clave se entregaría mediante variables de entorno o un almacén de secretos, como Azure Key Vault, nunca dentro del código.
- **Riesgo.** Quien conozca la clave puede firmar tokens válidos con cualquier rol; por eso no se muestra en capturas ni en este informe.

7. AUTORIZACIÓN POR ROLES Y PROPIETARIO

Se aplicaron dos niveles de autorización. El primero es por rol, con atributos en RegistrosController. El segundo es por propietario: RegistroService compara el identificador del usuario del token, obtenido mediante UsuarioActual, con el propietario guardado en el registro, y devuelve ResultadoOperacion.Prohibido cuando no coinciden.

Tabla 4. Matriz de permisos

Acción	Usuario	Admin
Crear cuenta e iniciar sesión	Permitido	Permitido
Consultar registros	Permitido	Permitido
Crear registros	Permitido	Permitido
Modificar un registro propio	Permitido	Permitido
Modificar un registro ajeno	403 Prohibido	403 Prohibido
Eliminar registros	403 Prohibido	Permitido

8. RESPUESTAS 401 Y 403

Las dos respuestas se devuelven en formato ProblemDetails mediante los eventos OnChallenge y OnForbidden de JwtBearer, implementados en RespuestasSeguridad. La diferencia es importante para el cliente:

- **401 No autenticado.** La API no sabe quién hace la petición: falta el token, la firma no coincide o el token venció. En el caso del vencimiento el detalle es «El token venció. Inicie sesión de nuevo.» y el encabezado WWW-Authenticate incluye error_description="The token expired". La aplicación borra la sesión y vuelve al inicio de sesión.
- **403 Prohibido.** La API sabe quién es el usuario, pero no tiene permiso para esa acción. El detalle es «No tiene permiso para realizar esta acción.» o «Solo puede modificar los registros que usted creó.». La aplicación muestra el mensaje y la sesión continúa.



Figura 2. Petición con token vencido desde curl: 401 y WWW-Authenticate con error_description

9. HTTPS Y DOCUMENTACIÓN EN SWAGGER

La API escucha en HTTPS por el puerto 7080 y en HTTP por el puerto 5080. UseHttpsRedirection redirige a HTTPS todas las peticiones, excepto las que llegan con el host 10.0.2.2, que es la dirección con la que el emulador de Android accede al equipo; esta excepción se configuró solo en appsettings.Development.json porque el emulador no confía en el certificado de desarrollo. En producción toda la comunicación debe usar HTTPS, ya que el token viaja en cada petición.

En Swagger se agregó el esquema de seguridad Bearer. El botón Authorize permite pegar el token y los endpoints protegidos muestran un candado.

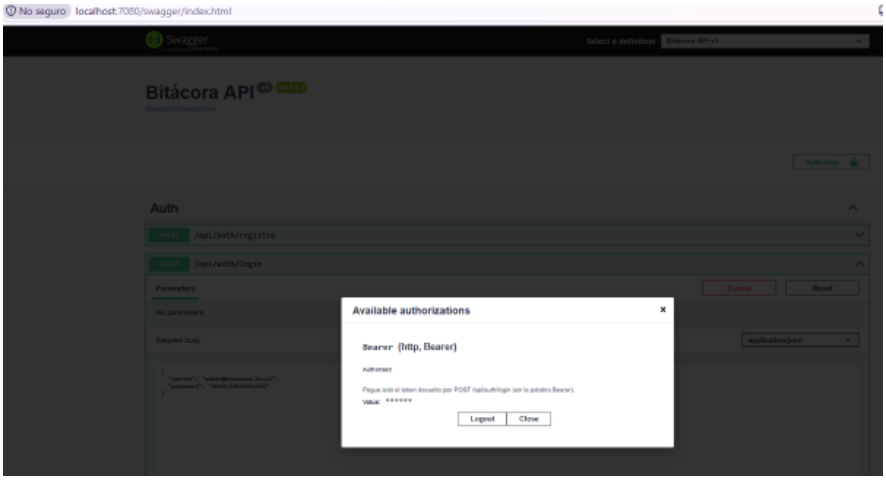


Figura 3. Ventana Available authorizations de Swagger con el esquema Bearer

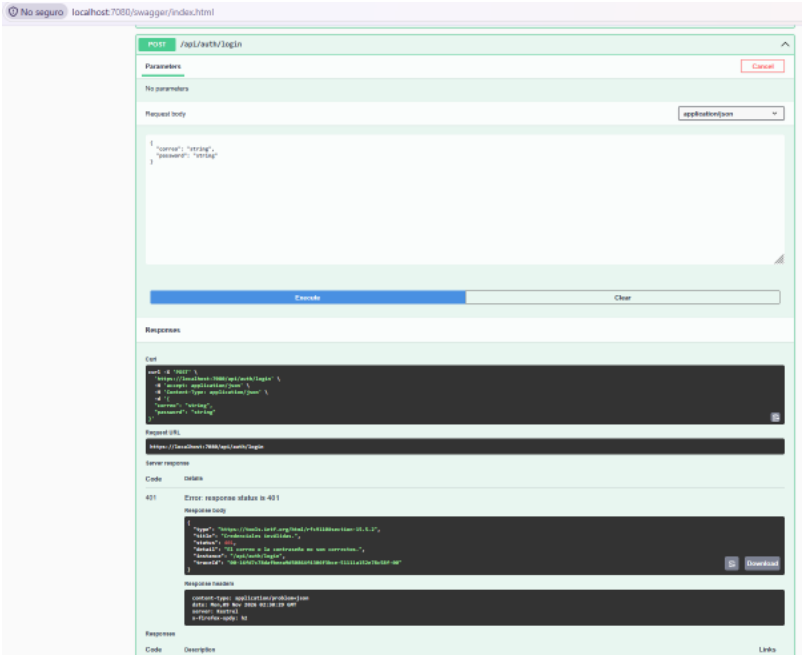


Figura 4. Swagger: inicio de sesión con credenciales incorrectas, 401

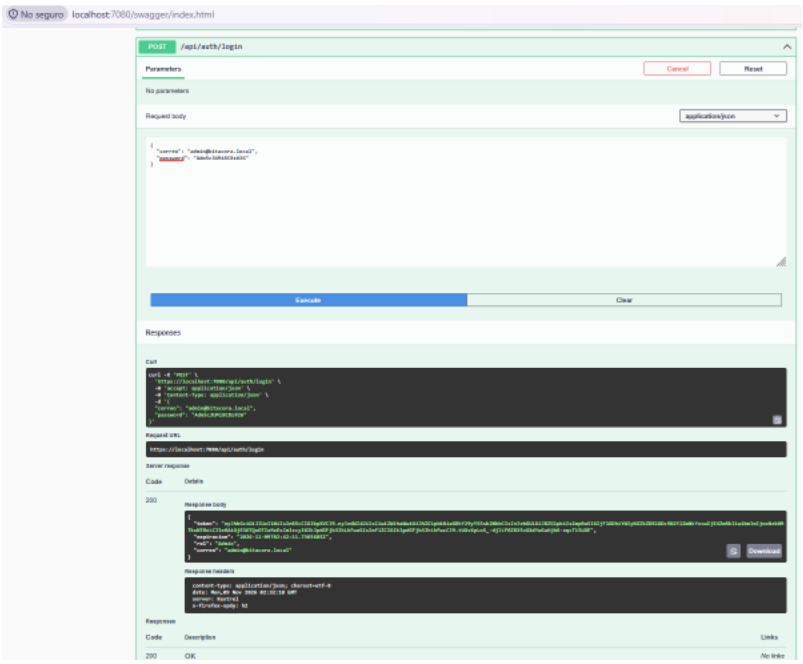


Figura 5. Swagger: inicio de sesión correcto, 200 con el token

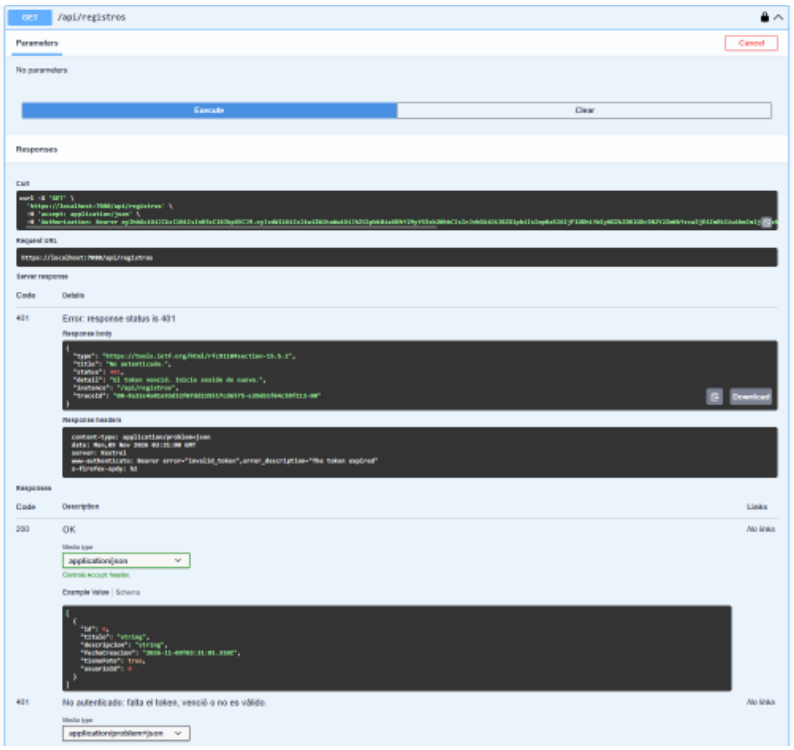


Figura 6. Swagger: GET /api/registros con token vencido, 401

10. SESIÓN EN LA APLICACIÓN ANDROID

La aplicación se adaptó manteniendo el patrón MVVM. LoginActivity es la pantalla de inicio; si ya existe una sesión vigente pasa directamente a la lista de registros.

- **SessionManager.** Guarda el token, su expiración, el rol y el correo en EncryptedSharedPreferences, de AndroidX Security, que cifra las claves con AES-256-SIV y los valores con AES-256-GCM usando una llave maestra del Android Keystore. Las reglas de respaldo excluyen este archivo de las copias de seguridad.
- **AuthInterceptor.** Interceptor de OkHttp que agrega el encabezado Authorization: Bearer con el token a todas las peticiones, sin repetir código en cada llamada.
- **Manejo de 401.** Cuando la API responde 401, o el token ya venció localmente, se borra la sesión y SesionActivity lleva al usuario al Login con el mensaje «Tu sesión expiró. Inicia sesión de nuevo.».
- **Manejo de 403.** ApiErrorMapper traduce la respuesta en un mensaje de permiso sin cerrar la sesión.
- **Interfaz por rol.** El botón Eliminar solo se muestra cuando el rol guardado es Admin, aunque la decisión final siempre la toma la API.
- **Cerrar sesión.** La opción del menú pide confirmación, borra la sesión cifrada y regresa al Login.

11. ANÁLISIS DEL TOKEN DECODIFICADO

La Figura 7 muestra el token del administrador decodificado en jwt.io. La cabecera y el contenido de un JWT solo están codificados en Base64URL, no cifrados; cualquiera puede leerlos, por eso el token no lleva contraseñas ni datos sensibles. Lo que impide modificarlo es la firma.

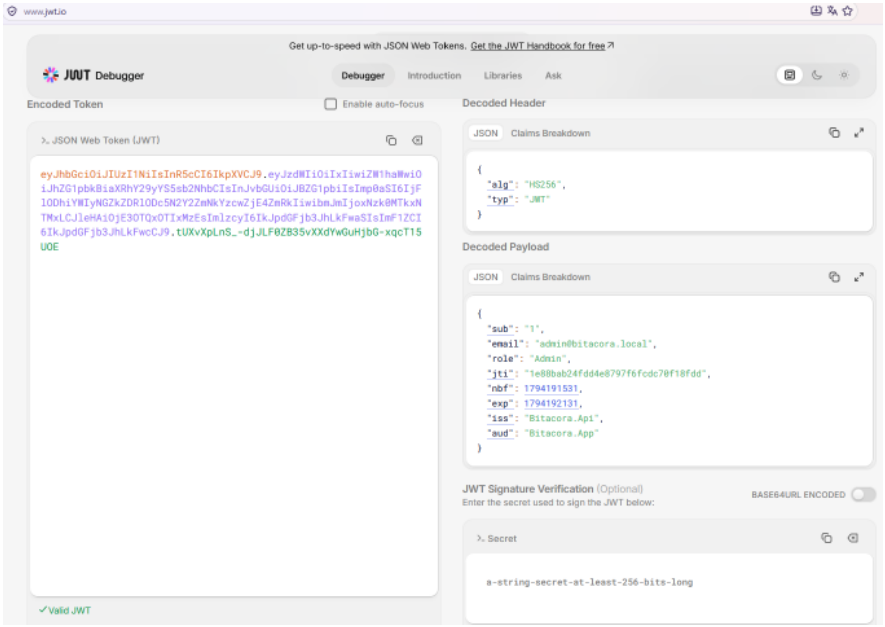


Figura 7. Contenido del token del administrador decodificado

Tabla 5. Claims del token

Claim	Valor observado	Significado
sub	Identificador del usuario	Sujeto del token; la API lo usa como propietario de los registros.

Claim	Valor observado	Significado
email	admin@bitacora.local	Correo del usuario autenticado.
role	Admin	Rol que evalúa [Authorize(Roles = "Admin")].
jti	Identificador único	Distingue cada token emitido.
nbf y exp	Fechas en segundos Unix	Inicio y fin de validez; exp menos nbf es 600 s, es decir, 10 minutos.
iss y aud	Bitacora.Api y Bitacora.App	Emisor y audiencia que la API valida.

Al cambiar cualquier carácter del token la firma deja de coincidir y la API responde 401, como se comprobó en la prueba 5a.

12. PRUEBAS DE SEGURIDAD

Las pruebas obligatorias se automatizaron en la carpeta «5 Pruebas de seguridad obligatorias» de la colección de Postman, con scripts que verifican el código de estado. Se ejecutaron con el entorno Bitácora - Local, donde las credenciales del administrador y el token vencido se guardan como variables secretas sin valor compartido.

Tabla 6. Resultado de las pruebas obligatorias

N.º	Prueba	Esperado	Resultado
1	Consultar registros sin token	401	401, «Debe iniciar sesión...»
2	Usuario intenta eliminar un registro	403	403, sin permiso
3	Admin elimina un registro	204	204 No Content
4	Usuario B modifica un registro del Usuario A	403	403, solo el propietario
5a	Token con la firma alterada	401	401
5b	Token vencido	401	401, «El token venció...»
6a	Login con contraseña incorrecta	401	401, mensaje genérico
6b	Login con correo inexistente	401	401, mismo mensaje genérico

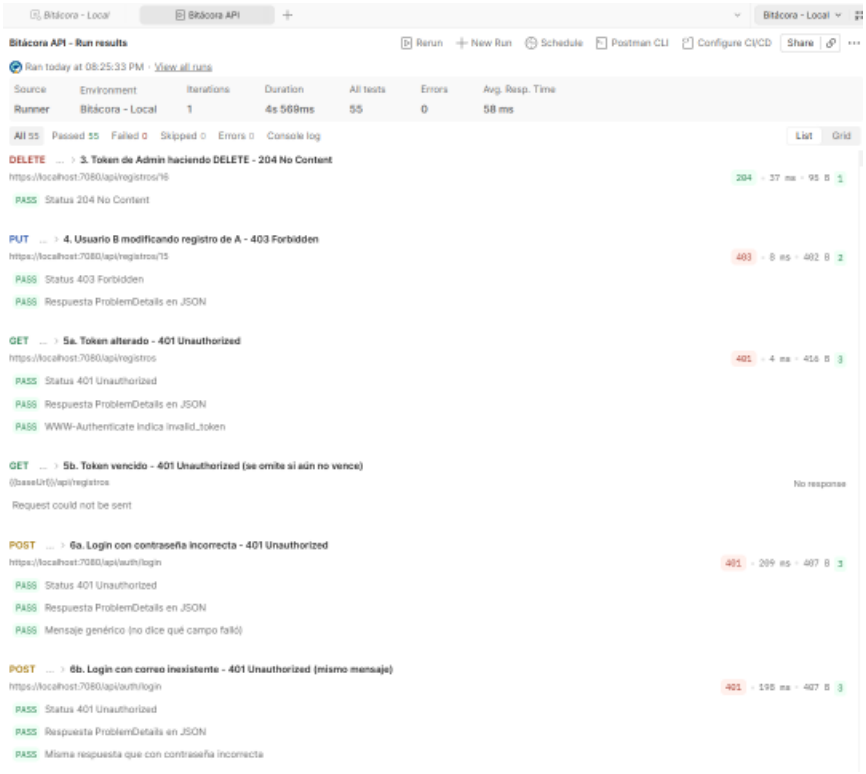


Figura 8. Ejecución de la colección: todas las pruebas aprobadas, 0 fallidas



Figura 9. Prueba 1: sin token, 401



Figura 10. Pruebas 2 y 3: Usuario recibe 403 y Admin elimina con 204



Figura 11. Prueba 4: modificar un registro ajeno, 403



Figura 12. Prueba 5: token alterado, 401

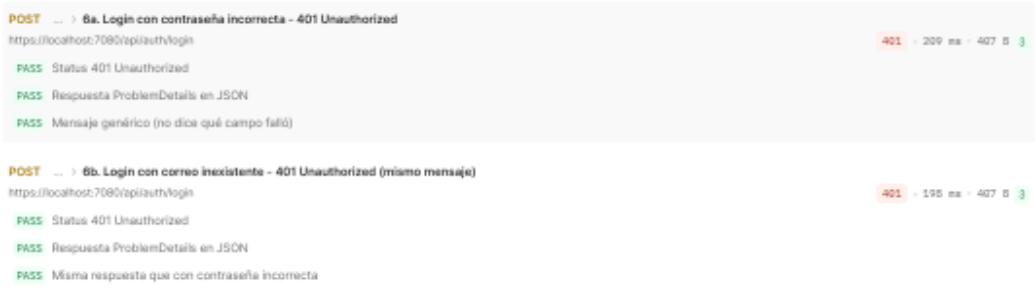


Figura 13. Prueba 6: credenciales incorrectas y correo inexistente, 401

El token vencido también se probó con curl (Figura 2) y desde la aplicación, que cerró la sesión al recibir el 401.

13. EVIDENCIAS DE LA APLICACIÓN

13.1 Inicio de sesión y creación de cuenta

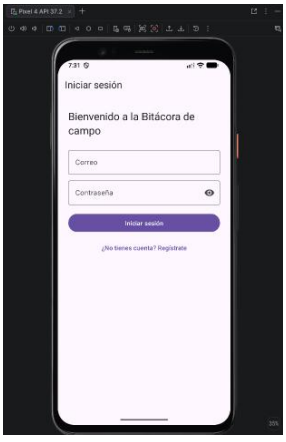


Figura 14. Pantalla de inicio de sesión

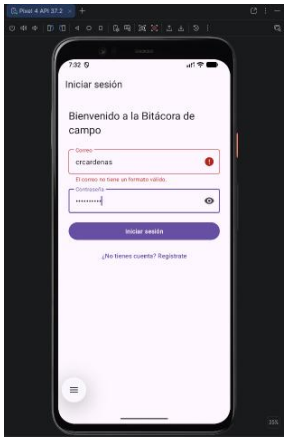


Figura 15. Validación de correo

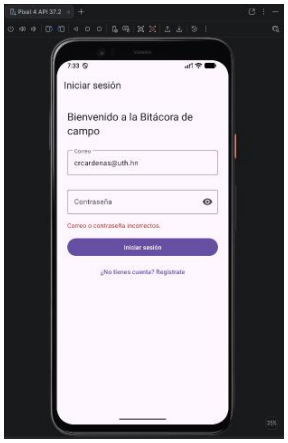


Figura 16. Credenciales incorrectas

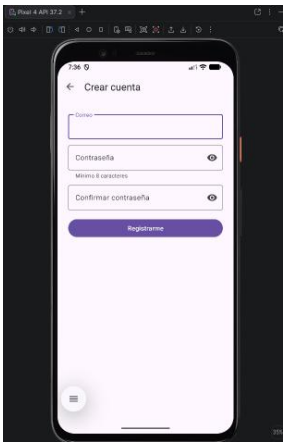


Figura 17. Crear cuenta

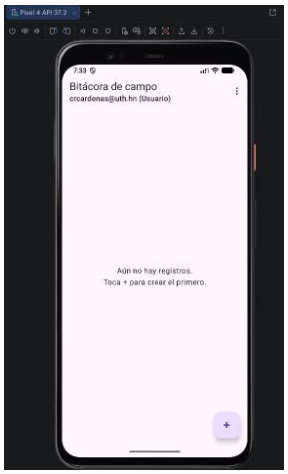


Figura 18. Lista del Usuario

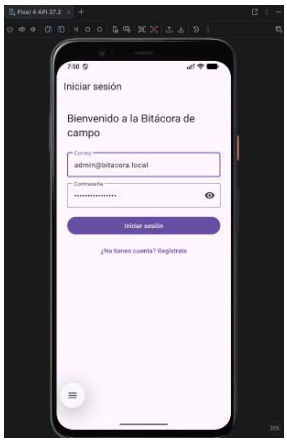


Figura 19. Inicio de sesión del Admin

13.2 Registros y permisos por rol

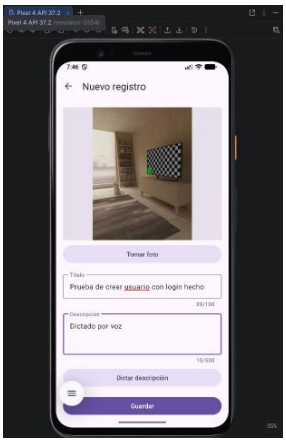


Figura 20. Nuevo registro con foto y voz

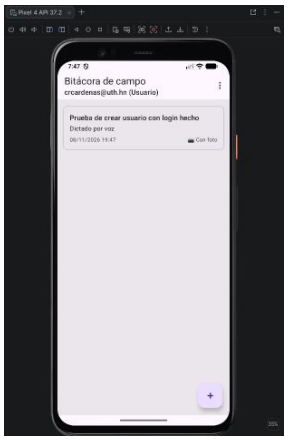


Figura 21. Lista con el registro

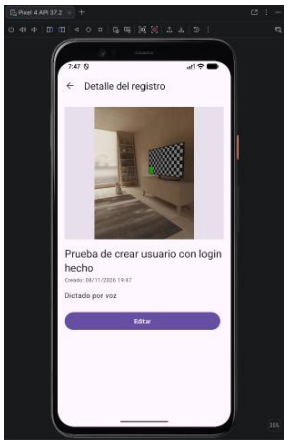


Figura 22. Detalle como Usuario, sin Eliminar

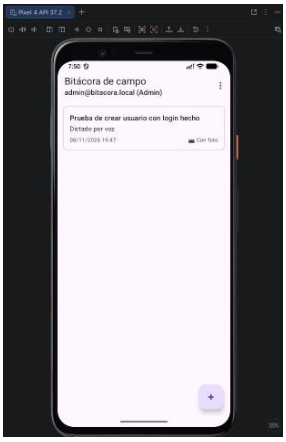


Figura 23. Lista como Admin

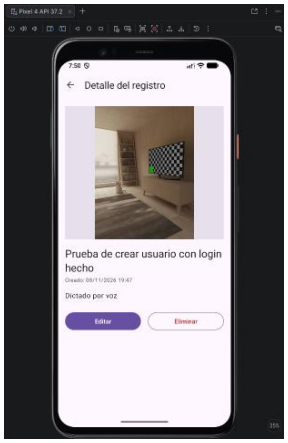


Figura 24. Detalle como Admin, con Eliminar

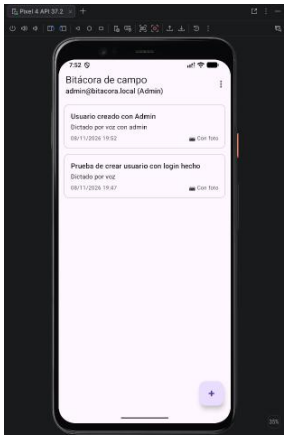


Figura 25. Registros de dos usuarios

13.3 Respuestas 403, sesión vencida y cierre de sesión

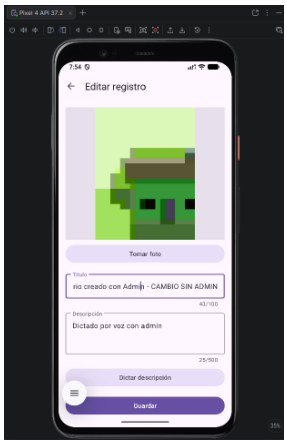


Figura 26. Edición de un registro ajeno

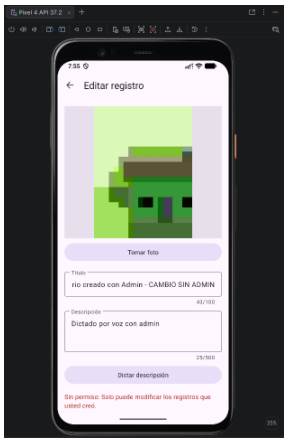


Figura 27. Respuesta 403 sin cerrar sesión

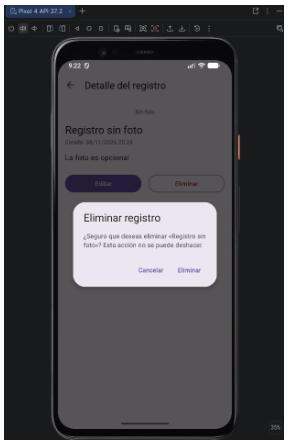


Figura 28. Sesión vencida, regreso al Login

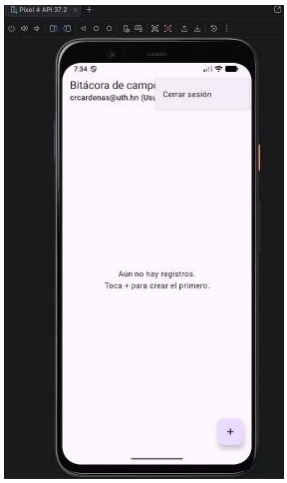


Figura 29. Menú Cerrar sesión

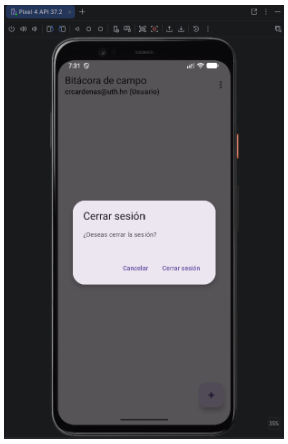


Figura 30. Confirmación de cierre

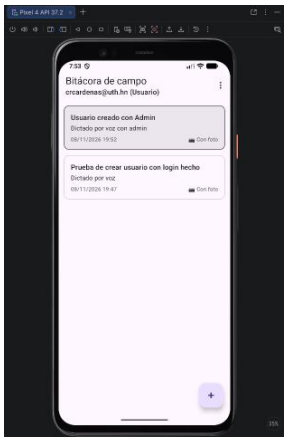


Figura 31. Lista del Usuario

14. OAUTH 2.0 CON CÓDIGO DE AUTORIZACIÓN Y PKCE

En esta tarea la propia API valida las credenciales y emite el token. En OAuth 2.0 esa responsabilidad se delega a un servidor de autorización externo, por ejemplo Google, Microsoft Entra ID o Keycloak, y la aplicación nunca recibe la contraseña del usuario.

Tabla 7. Participantes del flujo

Participante	En la bitácora
Propietario del recurso	La persona dueña de los registros, que inicia sesión y da su consentimiento.
Cliente	La aplicación Android, registrada como cliente público con un client_id.
Servidor de autorización	El proveedor que autentica al usuario y emite los tokens.
Servidor de recursos	La API de la bitácora, que valida el access_token y protege los registros.

Una aplicación móvil no puede guardar un secreto de cliente de forma segura, por eso se usa PKCE. Antes de iniciar, la app genera un code_verifier aleatorio y envía su hash SHA-256 como code_challenge. Si alguien intercepta el código de autorización no puede canjearlo, porque solo la app conoce el code_verifier original. El parámetro state protege contra falsificación de solicitudes. La Figura 32 muestra el flujo.

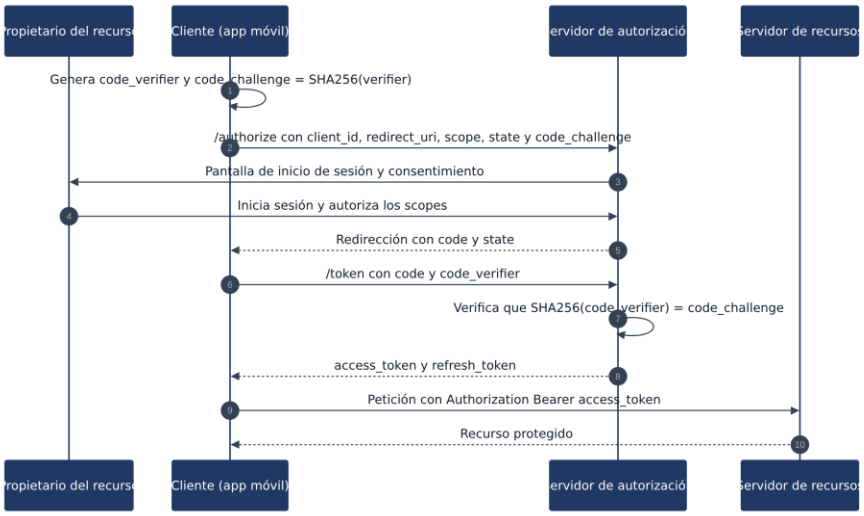


Figura 32. Flujo OAuth 2.0 con código de autorización y PKCE

Los scopes limitan lo que el token permite hacer. Para la bitácora podrían definirse bitacora.leer, bitacora.escribir y bitacora.eliminar, este último concedido solo a administradores, además de openid, profile y email si se usa OpenID Connect. La API validaría el scope además del rol.

Tabla 8. Comparación con la solución implementada

Aspecto	JWT propio (tarea 2.4)	OAuth 2.0 con PKCE
Quién valida la contraseña	La API de la bitácora	El servidor de autorización
La app conoce la contraseña	Sí, la envía al login	No, el usuario la escribe en el proveedor
Emisión del token	GeneradorTokenJwt	Servidor de autorización
Renovación	Nuevo inicio de sesión	refresh_token
Permisos	Rol en el claim role	Scopes y roles

15. DISTRIBUCIÓN DEL TRABAJO

El trabajo se dividió en seis partes, de modo que cada integrante aportó sus archivos en una rama propia y una solicitud de cambios revisada por otro miembro del grupo. La API se integró en orden (1, 2 y 3) porque cada parte depende de la anterior.

Tabla 9. Aporte de cada integrante

Integrante	Rama	Aporte
Michael Abad Díaz Castro	feature/api-usuarios	Entidades Usuario y Roles, propietario en Registro, hash de contraseñas, repositorio de usuarios y migración.
José Fernando Orellana Reyes	feature/api-jwt	DTOs de autenticación, AuthService, generación y validación del JWT, AuthController, Program.cs, Swagger Bearer y respuestas 401/403.
Romel Márquez	feature/api-autorizacion	Usuario actual, verificación del propietario, ResultadoOperacion y autorización por rol en RegistrosController.
Fernando Abad Diaz Castro	test/api-postman-seguridad	Colección y entorno de Postman con las pruebas obligatorias y documentación de la API.
Carlos Roberto Cárdenas Martínez	feature/android-sesion	SessionManager cifrado, AuthInterceptor, AuthApi, DTOs, repositorios, mapeo de errores y dependencia de seguridad.
Jennifer Mariela Márquez Chacón	feature/android-login-ui	ViewModels y pantallas de Login y Crear cuenta, manejo de 401/403 en la interfaz, menú de cierre de sesión, manifiesto y documentación.

16. CONCLUSIONES

- El token JWT con firma HS256, emisor, audiencia y vencimiento de diez minutos permite que la API identifique al usuario en cada petición sin guardar sesiones en el servidor.
- Separar la autorización por rol de la verificación del propietario evita que un usuario modifique o elimine información que no le pertenece, aunque tenga un token válido.
- Distinguir 401 de 403 permite que la aplicación reaccione correctamente: pedir un nuevo inicio de sesión o solo informar la falta de permiso.
- Guardar la clave en user-secrets y la sesión en EncryptedSharedPreferences protege los datos más sensibles tanto en el servidor como en el teléfono.
- OAuth 2.0 con PKCE sería el siguiente paso para delegar la autenticación a un proveedor externo sin que la aplicación maneje contraseñas.